Verification-Gated Recovery for Long-Horizon Language Agents

Anonymous Authors

August 30, 2026

Abstract

Verification for language agents is usually discussed as a score, critic, test, or process signal. In long-horizon execution, a second problem is equally important: what should the system *do* after verification fails? We present a package-resident protocol in which persistent intent is refined from Plan to Phase to a multi-view durable contract (DIC), instantiated through just-in-time execution stacks (EPS), and linked to evidence at prompt/run and action level. A verification gate returns one of four control outcomes—accept, local repair, upward replan, or material escalation—rather than only a scalar judgment. With complete dependency declarations and version-bound evidence, we prove that a repair need invalidate only its dynamic dependency cone, and that this scope is worst-case tight given footprint information alone. Under a conditional verifier miss bound α, fault survival across k observable gates is at most α^k; conversely, if the evidence channel aliases goal-satisfying and goal-violating states, no evidence-only verifier can avoid both false acceptance and false rejection. We instantiate the protocol in AI-Native Workflow and pre-register a five-task paired evaluation emphasizing iterative-spec regression, live stateful cutover, and scientific certification. The benchmark runs are pending; we therefore claim a verification/recovery mechanism and formal limits, not empirical superiority.

1 Introduction

Agents can now edit repositories, run tests, browse resources, call tools, and coordinate other agents. This makes verification both more valuable and more ambiguous. A unit test can reveal a failure, but it does not decide whether to retry the last command, invalidate several downstream artifacts, redesign the plan, or ask a human. A verifier may also check an artifact whose upstream assumptions have changed, allowing stale evidence to certify a now-invalid result.

Current agent work explores self-reflection [Shinn et al., 2023], process supervision [Lightman et al., 2023], scalable oversight [Irving et al., 2018, Leike et al., 2018, Burns et al., 2024], and security/evaluation environments [Ruan et al., 2024, Debenedetti et al., 2024]. We study a complementary control question: *how can verification be tied to persistent intent, artifact versions, dependencies, and an explicit recovery action?*

Our protocol stores a hierarchy

$$\text{Plan} \succ \text{Phase} \succ \text{DIC} \succ \text{EPS} \succ \text{Prompt/Run} \succ \text{Action}$$

inside the working package. The DIC is one multi-view contract for the whole Phase. Its Requests state what must be achieved; its Plan records prompt dependencies, parallel subagents, merge points, and conditional branches. EPS instances operationalize planned nodes against the current state. Evidence is linked to the object it purports to verify. Verification then produces a control outcome rather than merely a critique.

We make three contributions. First, we specify an executable verification/recovery state machine with evidence lineage and distinct local-repair, replan, and human-escalation semantics. Second, we prove a dependency-local recovery theorem and a matching worst-case necessity result under explicit completeness assumptions, plus simple imperfect-verifier and observability limits. Third, we pre-register a small resource-constrained evaluation that emphasizes regression and repair behavior rather than only terminal task success.

2 From verification signal to recovery decision

2.1 Persistent verification target

A verification call is meaningful only relative to a target claim and its authority. AI-Native Workflow therefore stores one durable intent contract (DIC) per Phase. `REQUESTS.md` is the authoritative inventory of goals and acceptance targets; `PLAN.md` records the execution graph; `TEST_MATRIX.md`, when present, maps software/theory/experiment cases to checks. The DIC can survive deletion and regeneration of the current execution stack.

A Phase's `reports/` directory is mandatory, flat, and capped at ten files by default. It contains a human report plus key merged raw outputs with provenance. Lossless collation is allowed; averaging, paraphrase, or removal of failed records is not considered raw evidence. This keeps the evidence surface compact without replacing the forensic log archive.

2.2 Four verification outcomes

Let D denote the DIC, s the current package/environment state and e the available evidence. The verification operator is

$$V(D, s, e) \in \{\mathsf{accept}, \mathsf{repair}, \mathsf{replan}, \mathsf{escalate}\}.$$

- **Accept**: the relevant request/phase has sufficient evidence.
- **Repair**: intent is unchanged; re-execute the smallest affected operational region.
- **Replan**: the durable execution design changes while the authorized objective remains intact.
- **Escalate**: machine authority is insufficient, e.g. material objective/scientific/protocol/release change, external/destructive action, credentials, or paid compute.

This separates verifier epistemics (what did the evidence show?) from recovery control (where should execution resume?).

2.3 Verification layers

Local verification is automatic and close to the producing action: compile, test, validate a schema, reopen an artifact, recompute a statistic. Milestone verification closes a larger Request or prompt batch. Fresh-context independent review is reserved for material claims or release boundaries. Thus a second agent is not mandatory after every step; independence is spent where correlated error is most consequential.

3 Dependency-local recovery

Let execution nodes form a DAG. Node v declares read keys R_v and writes W_v, and all possible data-flow dependencies are represented. If a repair changes keys Δ, define

$$B(\Delta) = \{v : R_v \cap \Delta \neq \varnothing\}, \qquad \mathcal{I}(\Delta) = \mathrm{Reach}(B(\Delta)).$$

Theorem 1 (Local recovery)**.** *Assume node outputs and evidence depend only on declared reads, predecessor outputs, recorded tool/version parameters, and recorded randomness; dependency edges are complete; and evidence is tied to artifact versions. After changing Δ, completed outputs outside $\mathcal{I}(\Delta)$ remain contract-valid.*

Sketch. A node outside the impact cone neither reads a changed key nor depends on a node that does. Topological induction preserves every declared input and therefore its previously verified output/evidence relation. The technical report gives the full statement and failure conditions. □

The impact cone is worst-case tight with respect to declared footprints. A direct reader can copy a changed bit, and every descendant can propagate it; therefore each reachable node can be made stale. This is an adaptation of dependency/incremental-build reasoning [Mokhov et al., 2018, Acar, 2009]. The operational value is not a new graph theorem but an explicit answer to "how much verified agent work must be reopened after this failure?"

Table 1: Five frozen candidate tasks; scores are pending.

ID	Task	Verification/recovery signal
T1	`env_manager`	5 progressive checkpoints; easy-control overhead
T2	`database_migration`	dependencies, rollback, old requirements remain active
T3	`recli`	8 checkpoints; regression/erosion under architectural growth
T4	`live-database-cutover`	continuous live-state consistency and safe cutover
T5	`certified-sparse-regression`	outcome certification in scientific optimization

The assumptions are essential. Hidden shared state, mutable external services, incomplete read sets, or evidence not bound to versions can make an apparently local repair globally unsafe. In such cases the correct verification outcome is replan/escalate or conservative re-verification rather than optimistic locality.

4 Imperfect verification

Suppose a persisting fault remains observable and the conditional probability that gate i misses it, given survival so far, is at most $\alpha < 1$. Then

$$\Pr[\text{survive } k \text{ gates}] \leq \alpha^k,$$

and expected detection delay is at most $1/(1-\alpha)$ gate intervals. This does not require independent misses, only the conditional bound. It motivates repeated evidence opportunities without implying that "more verifier calls" always fix a bad evidence channel.

Indeed, let $O(s)$ be the observation available to the verifier and $G(s)$ the relevant success predicate.

Proposition 1 (Evidence aliasing)**.** *If $O(s_+) = O(s_-)$ but $G(s_+) = 1$ and $G(s_-) = 0$, no verifier using only O can achieve both zero false acceptance and zero false rejection on these states.*

The proof is immediate: the verifier receives the same input and must return the same decision. The remedy is richer environment-grounded evidence, a weaker claim, or explicit uncertainty—not a more confident prompt.

5 Implementation and auditability

The reference runtime stores canonical state in `.ai-workflow/STATE.json` and Phase-local DIC/EPS/prompt/action records beneath the Plan path. Stable IDs and hashes reconstruct which execution produced which evidence. Optional verification/research/minor/checkpoint lanes can be materialized lazily; reports are always present. A fresh Main can recover the approved objective, active phase, unresolved escalations, and evidence pointers from the package alone.

Conformance tests cover multi-phase Plan authority, ordinary autonomous progression, periodic checkpoints, material escalation, package-only recovery, DIC custom views, parallel prompt nodes, report invariants, and compatibility surfaces. We classify this as implementation evidence only.

6 Targeted evaluation protocol

We deliberately use a small proof-of-concept design rather than a broad benchmark sweep. ChatGPT Plus is Main for every run. Claude Code / Pro A is the primary executor. We compare a competent flat condition F with the full workflow W; both can plan, iterate, and use task-local tests, but F omits the persistent DIC/EPS/request/invalidation state machine. Claude Pro B repeats only the two mechanism-critical tasks.

T1–T3 use SlopCodeBench's iterative-specification tasks [Orlanski et al., 2026]; T4 comes from Terminal-Bench [Merrill et al., 2026]; T5 from Terminal-Bench-Science [Terminal-Bench Science Team, 2026]. The core study is ten F/W paired runs with Pro A, plus four pre-declared F/W replications on T2/T3 with Pro B.

The primary quantities are official reward/pass, strict pass rate by SlopCodeBench checkpoint, regression/erosion, rework, repair scope, prompt/run count, Main turns, and wall time. We expect hierarchy may be pure overhead on

T1; the more diagnostic question is whether W retains earlier requirements and localizes repairs better on T2/T3. T4 tests stateful verification and premature completion; T5 probes transfer beyond software engineering.

With five tasks, we will show every run and paired/checkpoint traces rather than claim population-level significance. Oracle/reference solutions are used only for isolated environment qualification and are excluded from scored workspaces. At this draft stage, no scored run has occurred, and no empirical superiority claim is made.

7 Related work

Agent benchmarks such as AgentBench, WebArena, GAIA, SWE-bench, OSWorld, τ-bench, and Terminal-Bench expose failures in tool-mediated and stateful tasks [Liu et al., 2024, Zhou et al., 2024, Mialon et al., 2024, Jimenez et al., 2024, Xie et al., 2024, Yao et al., 2024, Merrill et al., 2026]. SlopCodeBench is especially relevant because requirements accumulate across checkpoints and regression is directly measured [Orlanski et al., 2026]. Reflection and process supervision provide feedback signals [Shinn et al., 2023, Lightman et al., 2023]; debate, recursive reward modeling, and weak-to-strong work address scalable oversight [Irving et al., 2018, Leike et al., 2018, Burns et al., 2024]. Our narrower focus is the control semantics after evidence arrives: how a verifier's result is tied to authority, dependency invalidation, repair scope, and persistent audit state.

8 Limitations

The locality theorem is only as sound as the dependency model. Hidden side effects and mutable external systems invalidate its assumptions. Verifier-error bounds do not model strategic reward hacking or correlated evidence failures beyond the stated conditional miss bound. The small experiment cannot establish general superiority and may be dominated by model-specific behavior of ChatGPT Main and Claude Code. Verification metadata also adds overhead and can encourage false confidence if the checks do not observe the real goal. Finally, the workflow is not a sandbox; security requires separate isolation and adversarial testing.

9 Conclusion

Reliable long-horizon agents need a response to verification failure, not only a verdict. By linking persistent requests, execution dependencies, versioned evidence, and four explicit recovery outcomes, AI-Native Workflow turns verification into a control loop whose scope can be audited. The formal results state exactly when local repair is safe and exactly why insufficient evidence cannot be repaired by hierarchy alone. The pending evaluation asks the pragmatic workshop question: does that extra structure reduce regression and rework enough to matter on real agent tasks?

References

Umut A. Acar. *Self-Adjusting Computation*. Morgan & Claypool, 2009.

Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. In *International Conference on Machine Learning*, 2024.

Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems*, 2024.

Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.

Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent alignment via reward modeling: A research direction. *arXiv preprint arXiv:1811.07871*, 2018.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=a7Qa4CcHak.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: A benchmark for general ai assistants. In *International Conference on Learning Representations*, 2024.

Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. In *Proceedings of the ACM on Programming Languages*, 2018.

Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Aws Albarghouthi, and Frederic Sala. Slopcodebench: Measuring code erosion under iterative specification refinement. *arXiv preprint arXiv:2603.24755*, 2026. URL https://arxiv.org/abs/2603.24755.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Toolemu: Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Terminal-Bench Science Team. Terminal-bench science: Evaluating ai agents on computational workflows in the natural sciences. https://github.com/harbor-framework/terminal-bench-science, 2026.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Tenyue Hua, Zhoujun Cheng, Dong-Suk Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.